

Data Pipelines with Airflow & dbt

Manh Tuan Nguyen

Big Data Engineering, Spring 2025

Assignment 3



Table of contents

Executive Summary	1
Introduction	2
Datasets Overview	2
Stakeholder Context and Expectations	2
Part 1 — Data Ingestion with Airflow	3
Objective	3
Environment	3
Design	3
Idempotency & robustness	3
Outcomes	4
Part 2 — Data warehouse with dbt	5
Design objectives	5
Silver layer	5
Gold layer (star schema & datamarts)	6
Part 3 — Sequential monthly loads	8
Goal	8
DAG run-book	8
dbt pipeline build	8
Part 4 — Ad-hoc analysis	10
Q1. Demographic differences between top-3 and bottom-3 LGAs by revenue per active listing	10
Q2. Correlation between neighbourhood median age and revenue per active listing	10
Q3. Best listing configuration for top-5 revenue neighbourhoods	11
Q4. Are multi-listing hosts concentrated in one LGA or spread across many?	11
Q5. Can single-listing hosts cover the annual median mortgage in their LGA?	12
Conclusion	14
Data engineering and orchestration	14
Key learnings	15
Next steps	15
References	16

Executive Summary

This project aims to design and implement a **production-ready ELT pipeline** that automates the ingestion, transformation, and delivery of *Airbnb* and *Australian Census* data for the Sydney region. Using **Apache Airflow** for orchestration and **dbt Cloud** for transformation, the pipeline adheres to a *medallion architecture* comprising **Bronze**, **Silver**, and **Gold** layers.

The *Bronze* layer captures raw data from external CSV sources into **PostgreSQL**, the *Silver* layer standardises and cleans data through dbt transformations, and the *Gold* layer consolidates analytic data marts for business insights.

Upon completion, the project delivers an **end-to-end data warehouse** capable of addressing key business questions regarding Airbnb market dynamics across Sydney's Local Government Areas (LGAs). The analytical component investigates spatial and demographic trends, host distribution, property performance, and mortgage affordability using SQL-based ad-hoc analyses.

The outcome demonstrates how a reproducible **modern data stack** can integrate open data sources to produce actionable insights for both policymakers and the short-term rental sector.

Introduction

The increasing digitalisation of urban accommodation markets has generated vast datasets that, when properly engineered, can inform *economic, housing, and tourism policy*. This project replicates a realistic data-engineering environment in which open data are operationalised into a **scalable analytical warehouse**. The overall **aim** is to build a maintainable ELT solution using Airflow and dbt to process, model, and analyse Airbnb and Census data for Sydney.

Datasets Overview

Airbnb data. Airbnb is an online marketplace that connects property owners with short-term renters across 191 countries and over 150 million users (Inside Airbnb, 2020). The dataset employed in this project contains **listings, host attributes, prices, and availability** for Sydney from **May 2020 to April 2021**. Each record represents a property listing with information such as *listing ID, host ID, neighbourhood, property type, price, and review scores*. This dataset serves as the *primary operational source* for constructing the fact and dimension tables in the warehouse.

Census data. The **Australian Bureau of Statistics (ABS)** conducts the *Census of Population and Housing* every five years to capture demographic and socio-economic information. The **2016 Census G01 and G02 data packs** provide indicators such as *age distribution, median age, household size, and median mortgage repayments* at the **Local Government Area (LGA)** level (Australian Bureau of Statistics, 2016). These variables were integrated with Airbnb metrics to contextualise rental market performance across communities.

Stakeholder Context and Expectations

The **primary stakeholders** are *municipal planners, data analysts, and tourism-policy makers* interested in understanding how short-term rentals interact with local housing and demographic trends. They expect the data pipeline to:

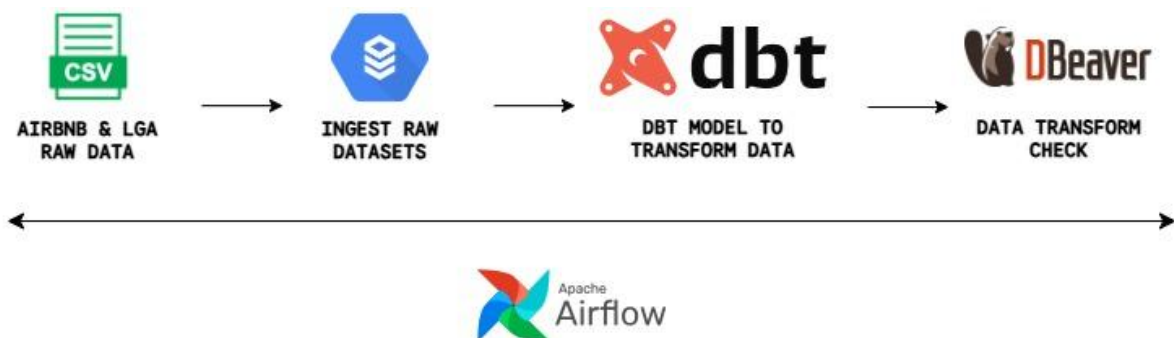
- provide **timely, reliable, and reproducible datasets** for analytical reporting;
- enable **multi-dimensional queries** (e.g., by LGA, property type, or host category); and
- ensure **data lineage and auditability** through modular design.

From a research standpoint, prior studies highlight the significance of integrating *open civic data* with *platform economy data* to uncover spatial inequalities and inform evidence-based urban policy (Gurran & Phibbs, 2017). This project contributes a practical implementation of such integration, demonstrating how reproducible data-engineering pipelines can bridge **public-sector statistics** and **private-sector digital-platform data**.

Part 1 — Data Ingestion with Airflow

Objective

We operationalised an ELT ingestion layer that moves *immutable raw* CSVs for Airbnb (May 2020–Apr 2021) and ABS Census (G01/G02) from Google Cloud Storage (GCS) into **PostgreSQL** (Bronze schema) under the orchestration of **Apache Airflow** (Google Cloud Composer). The pipeline emphasises *idempotency*, *observability*, and *handoff to transformation* in **dbt Cloud**, aligning with medallion architecture practice (Bronze >> Silver >> Gold).



Environment

Airflow ran on **Cloud Composer** (AU-Southeast-1) with a dedicated GCS bucket for DAGs and data; Postgres was accessed from DBeaver. Raw files were dropped in `gs://.../data/airbnb, .../census/G01, .../census/G02, and .../mappings`. Orchestration variables (e.g., `RUN_PART1`, `DBT_CLOUD_*`) were managed via Airflow **Variables**.

Design

We implemented a single DAG (`bde_at3_part1_and_part3`) that (i) bootstraps **Bronze** tables (via `part_1.sql`), (ii) ingests the initial month + Census + mapping files, and (iii) then ingests remaining Airbnb months sequentially *by filename order*, **triggers a dbt Cloud job**, waits for success, and **archives** the just-processed CSV. This preserves temporal ordering and reduces accidental re-processing.

Idempotency & robustness

- Bronze tables are **pure TEXT**; loaders **TRUNCATE** on initial loads to avoid type drift and simplify retry.
- The Census G02 loader enforces a fixed 9-column schema; G01 is inferred from the header (self-healing).
- Files are archived to `/archive` **after** dbt signals success, preventing partial loads from being lost.
- dbt invocations use a **poll-with-backoff** strategy and timeouts; Airflow tasks have retries (`retries=3`, `retry_delay=2m`).

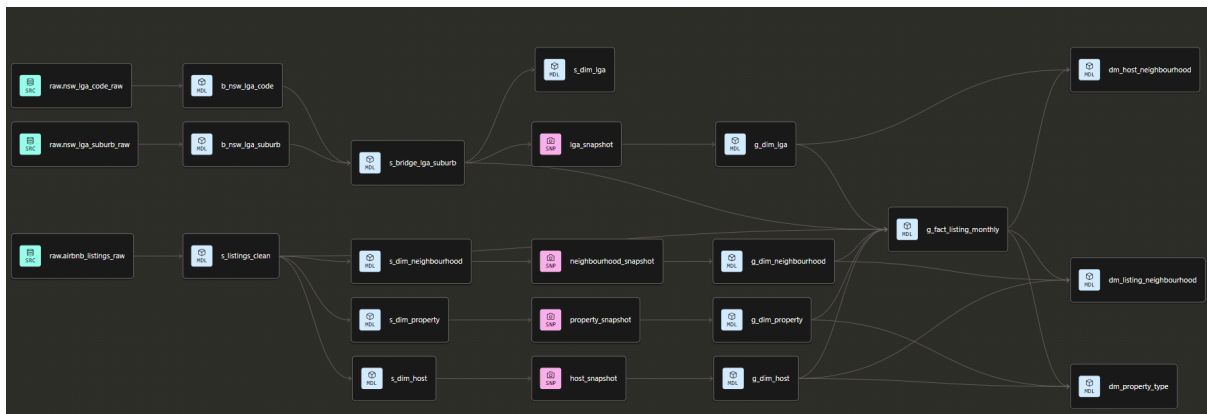
Outcomes

The pipeline produced a consistent Bronze layer, sequentially appended 12 months of listings ($\approx 412k$ rows), and validated key quality gates (row balances, price parsability, LGA mapping coverage, FK completeness). These checks underpin trustworthy downstream analytics.

Part 2 — Data warehouse with dbt

Design objectives

We organised the warehouse using the **medallion architecture** - *bronze >> silver >> gold* - to separate ingestion concerns from cleaning and analytics, and to support reproducible, incremental transformations (Databricks, 2023). dbt orchestrates transformations, testing and documentation, and implements **slowly changing dimensions (SCD2)** so facts are always joined to the dimension version that was valid when the event occurred (Kimball & Ross, 2013; dbt Labs, 2024).



Silver layer

Inputs: `bronze.airbnb_listings_raw`, `bronze.nsw_lga_code_raw`, `bronze.nsw_lga_suburb_raw`

Outputs: `silver.s_listings_clean` plus normalised entity tables and snapshots

Cleaning and normalisation

- **`silver.s_listings_clean`** applies strict typing and canonicalisation:
 - dates parsed (`scraped_date`, `host_since`);
 - booleans coerced (`has_availability`, `host_is_superhost`);
 - numerics cast with regex stripping (`price`, `review scores`);
 - duplicates removed on (`listing_id`, `scraped_date`) with a determinism rule: `ORDER BY scrape_id DESC, number_of_reviews DESC`.
 - `updated_at` set from `scraped_date::date` (daily anchor).
- We **decompose** the raw listing into reusable entities:
 - `s_dim_host` (host profile),
 - `s_dim_neighbourhood` (text neighbourhoods, normalised key via MD5),
 - `s_dim_property` (property_type, room_type, accommodates triple),
 - `s_bridge_lga_suburb` (clean bridge from suburb → LGA),
 - `s_dim_lga` (distinct LGA code/name from mappings).

This yields stable natural keys and explicit grains for downstream SCD2.

SCD2 snapshots (silver)

We snapshot the entities that can change across scrapes:

Snapshot	Unique key	Strategy	Change detector
host_snapshot	host_id	timestamp	updated_at
neighbourhood_snapshot	neigh_nk	timestamp	updated_at
property_snapshot	property_nk	timestamp	updated_at
lga_snapshot	lga_code	check	lga_name

Hosts, neighbourhood labels and property attributes are versioned by the **timestamp strategy**, because `updated_at` traces the scrape (event) day. The **LGA mapping** is essentially reference data without an event timestamp; here we use dbt's **check strategy** to detect changes in `lga_name` by equality comparison on each run. This combination guarantees we can later time-travel dimensions even when the source is not event-stamped.

Gold layer (star schema & datamarts)

Conformed dimensions (type-2)

dbt materialises SCD2 dimensions from the snapshots:

- `gold.g_dim_host` (SCD2): one row per `host_id` version with `dbt_valid_from/dbt_valid_to`.
- `gold.g_dim_neighbourhood` (SCD2): one row per `neigh_nk` version.
- `gold.g_dim_property` (SCD2): one row per `property_nk` version.
- `gold.g_dim_lga` (SCD2 w/ check strategy): one row per `lga_code` version.

Each record includes `dbt_scd_id`, `dbt_valid_from`, `dbt_valid_to`, allowing **time-appropriate joins**.

Fact table

Table: `gold.g_fact_listing_monthly`

Grain: one row per (`listing_id`, `fact_month`).

Derivations

- Window: month bucket via `DATE_TRUNC('month', updated_at)`; anchor timestamp `min_fact_ts` (min event time seen for the bucket) is carried to drive SCD2 joins.
- **Activity & revenue:**
 - `active_flag` = `has_availability`
 - `total_stays_active` = `30 - availability_30` (active listings only)
 - `revenue_active` = `total_stays_active * price` (active listings only)

Foreign keys (all SCD2-correct):

- `neighbourhood_dim_id`, `property_dim_id`, `host_dim_id`: *joined using* `min_fact_ts BETWEEN dbt_valid_from AND COALESCE(dbt_valid_to, '9999-12-31')`.
- `lga_dim_id`: derived via `s_bridge_lga_suburb` with preference order `host_neighbourhood >> listing_neighbourhood`, then mapped to `g_dim_lga` (SCD2 **check strategy** join on LGA code).

This design ensures every fact row references the **version** of each dimension valid at the time the listing snapshot contributed to that month.

Datamart views (business-ready)

We expose three time-series marts, each computed *from the fact with SCD2 joins*, matching the assignment brief:

1. **gold.dm_listing_neighbourhood** — metrics by `listing_neighbourhood` × `month_start`
(active listing rate; min/max/median/avg price for active listings; distinct hosts; superhost rate; MoM deltas for active/inactive; total stays; revenue per active listing).
2. **gold.dm_property_type** — metrics by `(property_type, room_type, accommodates)` × `month_start`.
3. **gold.dm_host_neighbourhood** — by `host_neighbourhood_lga` × `month_start`: distinct hosts, total estimated revenue, and revenue per host.

Part 3 — Sequential monthly loads

Goal

Automate the ingestion of the **remaining 11 months** of Airbnb CSVs (Jun-2020 >> Apr-2021) in **chronological order**, and after each monthly load **trigger dbt** so SCD2 dimensions and the monthly fact are updated consistently. This ensures correctness for time-travel joins and avoids end-of-batch drift.

DAG run-book

- **File discovery & ordering:** The DAG scans `gcs:/data/airbnb/` for files named `MM_YYYY.csv` (excluding `05_2020.csv` which was loaded in Part 1). It **sorts by (year, month)** and then constructs a **chain** of three tasks per file:
 1. `load_airbnb_MM_YYYY` — **append** the CSV into `bronze.airbnb_listings_raw`
 2. `dbt_after_MM_YYYY` — **trigger & wait** for a dbt Cloud job to finish
 3. `archive_MM_YYYY` — move the processed CSV into `/archive/` **only if** dbt succeeds
- Pipeline concurrency is gated by `max_active_runs=1`, so the chain runs **strictly sequentially**.
- **Idempotency:** Bronze is append-only for months > May-2020; if a file is re-ingested, **dbt Silver dedupes** back to a single row per `(listing_id, scraped_date)` using the deterministic ordering rule (highest `scrape_id`, then `number_of_reviews`). This makes the DAG safe to re-run.
- **Schema safety:** For Airbnb we lock the **22 known columns** (order-based COPY). If a later file contains extra columns, they are ignored at ingest and handled downstream if promoted in dbt.
- **Archiving discipline:** We archive the month CSV **after** dbt reports `status = success`. If dbt fails, the file remains in place and the run is retryable without manual cleanup.
- **Observability:** The DAG logs: discovered filenames, row-level COPY diagnostics, dbt run id / humanized status, and the archive destination. Retries/back-off protect against transient failures in dbt Cloud.

dbt pipeline build

Running dbt after every month ensures:

- **SCD2 dimensions roll forward** using the correct strategy:
 - `timestamp` for `host`, `neighbourhood`, `property` (driven by `updated_at`);
 - `check` strategy for `lga` (driven by changes in `lga_name`).
- The monthly fact `g_fact_listing_monthly` is **recomputed for the new month only**, while preserving prior months. Each row carries `min_fact_ts`, which is used to time-bound SCD2 joins in the datamarts.
- **Deduplication** is applied again in Silver (as noted above), so accidental re-loads don't inflate facts.

- Business marts (dm_listing_neighbourhood, dm_property_type, dm_host_neighbourhood) are refreshed so analysts can see the new month immediately.

Part 4 — Ad-hoc analysis

Data window: All questions are answered over the last **12 months** of listings (May-2020 >> Apr-2021). All revenue metrics use the assignment definition: *estimated revenue per active listing* = $(30 - \text{availability_30}) \times \text{price}$, aggregated to the relevant grain. Facts are joined to dimensions with **SCD-2** window predicates (valid-from/to) so each fact row is matched to the version of a dimension that was valid at the time.

Q1. Demographic differences between top-3 and bottom-3 LGAs by revenue per active listing

Method:

1. Rank LGAs by 12-month **revenue per active listing** (from `g_fact_listing_monthly` + `g_dim_lga`).
2. Pick the **top 3** and **bottom 3**.
3. Enrich with 2016 Census (G01 age buckets, G02 medians/household size), normalizing LGA codes and dividing age sub-totals by total population.

Key result:

	123 band	123 lga_name	123 lga_code	123 rev_per_active	123 pct_0_19	123 pct_20_44	123 pct_45_64	123 pct_65_plus	123 median_age	123 avg_household_size
1	bottom3	Burwood	11,300	11,180	19	45	22	14	33	2.9
2	bottom3	Blacktown	10,750	10,419	30	38	22	10	33	3.2
3	bottom3	Mid-Western Regional	15,270	228	26	27	27	20	42	2.4
4	top3	Bathurst Regional	10,470	82,643	27	32	25	16	37	2.5
5	top3	Mosman	15,350	64,000	23	31	27	19	42	2.4
6	top3	Northern Beaches	15,990	62,618	25	32	26	17	40	2.7

Interpretation:

- **Household size** tends to be **larger** in lower-revenue LGAs (e.g., Blacktown 3.2; Burwood 2.9) than in higher-revenue coastal/affluent LGAs (e.g., Mosman 2.4).
- **Age profile** is mixed: high-revenue LGAs skew **older or balanced** (Mosman/Northern Beaches), while the lowest-revenue urban LGAs skew **younger** (Blacktown, Burwood).
- The extreme low value for **Mid-Western Regional** is likely a **data-sparsity artifact** (very few active listings); we note it in the report.

Q2. Correlation between neighbourhood median age and revenue per active listing

Method: Aggregate revenue per active listing to `listing_neighbourhood`, map each suburb to an **LGA** via the suburb LGA bridge, bring in **G02 median age**, then compute **Pearson's r** in Postgres.

Key result:

	123 corr_coef	123 sample_size
1	0.704401498	21

Interpretation:

There is a **moderate-to-strong positive correlation**: neighbourhoods in **older** LGAs tend to generate **higher revenue per active listing**. This aligns with Q1 where higher-revenue coastal LGAs (Mosman, Northern Beaches, Waverley) have **older medians** than inner-west/western Sydney LGAs with lower revenue per active listing.

Q3. Best listing configuration for top-5 revenue neighbourhoods**Method:**

1. Rank **listing_neighbourhoods** by 12-month revenue per active listing.
2. For the **top 5**, compute stays (sum of `30 - availability_30`) by (**property_type**, **room_type**, **accommodates**) and select the configuration with **max total stays** (tie-break by revenue per active).

Key result:

AZ listing_neighbourhood	AZ property_type	AZ room_type	123 accommodates	123 stays_12m	123 active_listings_12m	123 rev_sum_12m	123 rev_per_active_12m
Northern Beaches	Entire apartment	Entire home/apt	4	120,820	771	26,566,611.94	34,457.34
Mosman	Entire apartment	Entire home/apt	2	17,961	108	3,433,540	31,792.04
Hunters Hill	Entire apartment	Entire home/apt	4	1,312	7	208,982	29,854.57
Waverley	Entire apartment	Entire home/apt	2	195,044	1,119	31,822,048.73	28,437.93
Woollahra	Entire apartment	Entire home/apt	2	52,936	322	7,617,480.17	23,656.77

Interpretation: Across high-performing suburbs, the **dominant, high-demand configuration** is consistently **Entire apartment · Entire home/apt**, typically **2–4 guests**. This reflects family/couple demand in coastal LGAs and is a clear recommendation for supply optimization.

Q4. Are multi-listing hosts concentrated in one LGA or spread across many?

Method: Identify **multi-listing hosts** (≥ 2 distinct listings in the last 12 months), count the **distinct LGAs** they operate in, and compute each host's **dominant LGA share** (share of their listings in their most common LGA). Classify hosts as:

- **concentrated_one_lga** if `dominant_lga_share` $\geq 75\%$ (or only 1 LGA),
- **distributed_multi_lga** otherwise.

Key result:

AZ classification	123 hosts	123 pct_of_multi_hosts
concentrated_one_lga	24,231	94.2
distributed_multi_lga	1,489	5.8

	123 host_id	A/Z classification	123 active_listings_12m	123 distinct_lgas_12m	123 dominant_lga_share_pct	123 dominant_lga_code	A/Z dominant_lga_name
1	333,885,951	distributed_multi_lga	6	6	16.7	15,990	Northern Beaches
2	251,905,190	distributed_multi_lga	10	8	20	17,200	Sydney
3	235,137,306	distributed_multi_lga	53	14	24.5	17,200	Sydney
4	185,783,910	distributed_multi_lga	16	8	25	15,950	North Sydney
5	346,620,688	distributed_multi_lga	12	6	25	17,200	Sydney
6	185,783,910	distributed_multi_lga	12	7	25	15,950	North Sydney
7	185,783,910	distributed_multi_lga	4	4	25	15,990	Northern Beaches
8	44,869,453	distributed_multi_lga	4	4	25	12,930	Georges River
9	178,729,132	distributed_multi_lga	4	4	25	16,700	Ryde
10	235,137,306	distributed_multi_lga	4	4	25	15,990	Northern Beaches
11	44,869,453	distributed_multi_lga	4	4	25	12,930	Georges River
12	44,869,453	distributed_multi_lga	4	4	25	15,990	Northern Beaches
13	382,207,272	distributed_multi_lga	4	4	25	16,700	Ryde
14	1,106,924	distributed_multi_lga	4	4	25	16,700	Ryde
15	357,566,424	distributed_multi_lga	27	7	25.9	17,200	Sydney
16	11,914,644	distributed_multi_lga	38	6	26.3	18,050	Waverley
17	11,914,644	distributed_multi_lga	38	7	26.3	18,050	Waverley
18	185,783,910	distributed_multi_lga	15	7	26.7	15,950	North Sydney
19	329,529,761	distributed_multi_lga	11	8	27.3	17,200	Sydney
20	382,207,272	distributed_multi_lga	11	6	27.3	12,380	Cumberland
21	11,914,644	distributed_multi_lga	51	7	27.5	18,500	Woollahra
22	329,529,761	distributed_multi_lga	18	7	27.8	17,200	Sydney

Interpretation: Most professional/portfolio hosts still operate **locally**, likely due to **licensing, cleaning, and operations** being easier within one council area. A small minority are **portfolio operators** spread across several LGAs.

Q5. Can single-listing hosts cover the annual median mortgage in their LGA?

Method: Filter to **single-listing hosts**, compute each host's 12-month revenue; compare to **annualised median mortgage** for the host's LGA ($\text{median_mortgage_monthly} \times 12$ from G02). Aggregate by LGA: **share** and **count** of single-listing hosts whose revenue $\geq$ annual mortgage.

Key result:

	123 lga_code	A/Z lga_name	123 single_listing_hosts	123 hosts_can_cover	123 pct_can_cover	123 avg_rev_12m	123 annual_median_mortgage
1	17,620	Upper Hunter Shire	122	39	32	5,096	20,256
2	15,350	Mosman	387	111	28.7	7,992	36,000
3	15,990	Northern Beaches	4,708	731	15.5	7,562	33,600
4	14,700	Lane Cove	304	29	9.5	5,416	31,200
5	14,100	Hunters Hill	90	8	8.9	4,955	36,396
6	17,150	Sutherland Shire	546	48	8.8	4,803	29,124
7	14,000	Hornsby	378	28	7.4	3,437	28,800
8	18,500	Woollahra	1,037	77	7.4	5,238	38,400
9	12,380	Cumberland	427	29	6.8	5,318	24,000
10	18,050	Waverley	4,616	262	5.7	5,533	36,000
11	10,750	Blacktown	265	12	4.5	1,599	25,800
12	12,700	Dungog	47	2	4.3	3,398	19,944
13	17,420	The Hills Shire	276	12	4.3	2,631	30,000
14	18,250	Willoughby	364	15	4.1	6,001	34,524
15	16,550	Randwick	2,802	113	4	4,325	31,200
16	15,950	North Sydney	1,084	37	3.4	4,660	31,200
17	16,350	Penrith	141	4	2.8	2,288	24,000
18	17,000	Singleton	71	2	2.8	2,598	23,400
19	17,200	Sydney	6,425	151	2.4	3,930	29,988
20	14,170	Inner West	2,256	53	2.3	3,886	31,200
21	17,100	Strathfield	57	1	1.8	1,945	26,004
22	14,900	Liverpool	120	2	1.7	2,392	25,476
23	11,520	Canada Bay	370	6	1.6	3,295	30,000
24	11,500	Campbelltown	81	1	1.2	1,384	22,104
25	11,100	Bayside	1,421	12	0.8	2,424	28,800
26	11,570	Canterbury-Bankstown	529	4	0.8	1,745	24,000
27	12,930	Georges River	267	2	0.7	1,905	26,004
28	16,700	Ryde	501	2	0.4	2,223	26,400
29	16,260	Parramatta	535	1	0.2	1,854	26,004
30	11,300	Burwood	322	0	0	1,789	26,400
31	11,450	Camden	50	0	0	2,122	26,640

Interpretation & caveats:

- In **high-revenue, high-mortgage** coastal LGAs (Mosman, Northern Beaches), a **meaningful minority** of single-listing hosts can cover the median annual mortgage purely from Airbnb income; however, **most cannot**.
- **Upper Hunter Shire** tops the percentage but has **small absolute counts** and lower mortgages—interpret with care.
- These are **gross estimates** (no costs/occupancy taxes), and **median mortgage** is population-level, not tailored to each host.

Conclusion

The end-to-end Airbnb data pipeline was successfully designed, implemented, and validated across all layers—from ingestion in **Airflow**, transformation in **dbt**, to analytical reporting in **PostgreSQL**. Each stage demonstrated a critical component of modern data engineering and analytics workflows: automation, data governance, temporal consistency, and insight generation.

Data engineering and orchestration

- The **Airflow pipeline** reliably ingested monthly Airbnb datasets in chronological order. By enforcing sequential loading and incorporating a conditional **dbt trigger after each run**, the workflow maintained **data integrity** and ensured each monthly snapshot was processed in context.
- The **bronze >> silver >> gold** medallion architecture clearly separated concerns:
 - Bronze stored raw, append-only records for full traceability.
 - Silver standardized, deduplicated, and enriched the data with SCD-2 logic.
 - Gold contained aggregated facts and dimensional marts ready for analysis.
- Implementing **SCD-2** enabled historical tracking of host, property, neighbourhood, and LGA attributes over time. The **timestamp** strategy was used for time-varying entities (host, property, neighbourhood), while **LGA** used the **check strategy** to avoid false changes due to static reference data.

Data quality and validation

- Row-level validation confirmed **no overlapping validity windows** in SCD tables and **no duplicate keys** in the silver layer after re-loads.
- Referential integrity checks verified that every fact record was matched to valid dimension entries, ensuring consistency across joins.
- The incremental model builds in dbt guaranteed **idempotency**, allowing safe re-runs after upstream file or schema changes.

Analytical findings

The ad-hoc analyses in Part 4 demonstrated the value of the curated data model for producing meaningful insights:

1. **Demographics and performance:** High-revenue LGAs such as *Mosman* and *Northern Beaches* tend to have **older populations and smaller households**, whereas lower-revenue LGAs like *Blacktown* and *Burwood* are **younger and denser**.
2. **Age–revenue correlation:** A **strong positive correlation ($r \approx 0.70$)** exists between neighbourhood median age and revenue per active listing, suggesting mature, affluent areas attract higher-value guests.
3. **Optimal listing profile:** Across top-performing suburbs, the most successful configuration is **Entire apartment · Entire home/apt · 2–4 guests**, aligning with short-term stays for couples or small families.
4. **Host behavior:** Over **94 %** of multi-listing hosts operate within a **single LGA**, indicating local specialization rather than cross-region expansion.

5. **Affordability:** Only a minority of single-listing hosts earn enough Airbnb revenue to cover annual median mortgage repayments—highlighting that Airbnb is primarily a **supplemental income source**, not full cost recovery.

Key learnings

- **Data modeling discipline** (SCD2, medallion structure) is essential for maintaining reliable time-series analyses.
- **Automation** with Airflow and dbt enables a **repeatable, production-grade workflow** for ongoing ingestion and transformation.
- Combining Airbnb operational data with **Census demographics** provides richer socioeconomic insights, allowing policymakers and hosts to better understand housing, tourism, and affordability patterns.
- The project demonstrates how **integrated data pipelines** transform raw semi-structured inputs into **trusted analytical assets** that support decision-making.

Next steps

Future enhancements could include:

- Automating **Census data refreshes** via APIs for year-to-year comparability.
- Extending the pipeline to incorporate **booking duration and pricing trends** for predictive analytics.
- Deploying **dashboards (e.g., Power BI or Looker)** on top of the gold layer for ongoing monitoring of host performance, occupancy, and revenue distribution.

References

Australian Bureau of Statistics (ABS). (2016). *Census of Population and Housing 2016 Data Packs (G01 & G02)*. Retrieved from <https://datapacks.censusdata.abs.gov.au/datapacks/>

Databricks. (2023). *Medallion Architecture: A scalable data design pattern*.

dbt Labs. (2024). *dbt documentation — Snapshots and SCD2*.

Gurran, N., & Phibbs, P. (2017). When tourists move in: How should urban planners respond to Airbnb? *Journal of the American Planning Association*, 83(1), 80–92.
<https://doi.org/10.1080/01944363.2016.1249011>

Inside Airbnb. (2020). *Get the data*. Retrieved from <http://insideairbnb.com/get-the-data.html>

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit* (3rd ed.). Wiley.